

第二章

函数、类与模块化

Lianghai Xiao

<https://github.com/styluck/mlb>

学习目标

- 用“输入—处理—输出”设计函数；
- 正确使用位置参数、关键字参数和默认参数；
- 知道何时使用类，何时普通函数已经足够；
- 将代码组织成模块和包；
- 完成一个可导入、可测试的数据读写工具。

实验：

读取上海、深圳两个市场的 CSV 数据，合并后按日期筛选，并将结果写回文件。

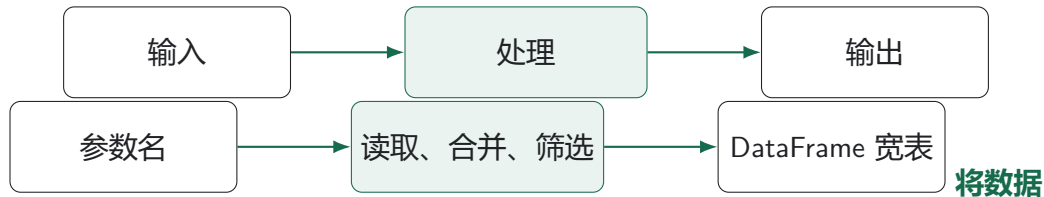
经典问题：写了多行同样的代码

```
close_sh = pd.read_csv("close_sh.csv", index_col=0)
close_sz = pd.read_csv("close_sz.csv", index_col=0)
close = pd.concat([close_sh, close_sz], axis=1)

open_sh = pd.read_csv("open_sh.csv", index_col=0)
open_sz = pd.read_csv("open_sz.csv", index_col=0)
open_ = pd.concat([open_sh, open_sz], axis=1)
```

代码做同样的事，只改了载入文件名。可读性差、扩展性差。

函数的思考方式



处理过程函数化，可读性和扩展性都能提升。

例：载入某段时间内的数据

```
def select_period(data, start_date, end_date):  
    """Return rows within a closed date interval."""  
    start = pd.to_datetime(start_date)  
    end = pd.to_datetime(end_date)  
  
    if start > end:  
        raise ValueError("start_date must not exceed end_date")  
  
    return data.loc[start:end]
```

- 函数的输入: `data`, `start_date`, `end_date` (不等于数据的输入);
- 函数的输出: `return` 命令后面的变量。
- 文档字符串: 说明函数的功能。

return 不等于 print

只显示，不交回结果

```
def add(a, b):  
    print(a + b)
```

```
x = add(2, 3)  
# x is None
```

返回后可继续计算

```
def add(a, b):  
    return a + b
```

```
x = add(2, 3)  
y = x * 10
```

位置参数、关键字参数与默认值

```
def data_load(field, start_date, end_date,
              data_dir="dataset"):
    ...

# positional arguments
close = data_load("close", "2020-01-01", "2020-12-31")

# keyword arguments: easier to read
close = data_load(
    field="close",
    start_date="2020-01-01",
    end_date="2020-12-31",
)
```

默认值应适合大多数使用场景，必要时仍允许显式覆盖。

函数接口约定

```
def data_load(  
    field: str,  
    start_date: str,  
    end_date: str,  
    data_dir: str | Path = "dataset",  
) -> pd.DataFrame:  
    """Load SH/SZ data in the requested date interval."""
```

- 类型标注帮助阅读和编辑器检查,
- 如果类型错误会自动返回 error, 但不会拦截逻辑错误输入。

输入逻辑错误

```
def _parse_period(start_date, end_date):  
    start = pd.to_datetime(start_date, errors="raise")  
    end = pd.to_datetime(end_date, errors="raise")  
  
    if start > end:  
        raise ValueError(  
            "start_date must not exceed end_date"  
        )  
    return start, end
```

Tips:

错误要在离输入最近的地方被发现，错误信息应说明“哪里错了”，以方便我们后续 debug。

内置函数

```
def _read_market(field, market, data_dir):  
    path = Path(data_dir) / f"{field}_{market}.csv"  
    if not path.exists():  
        raise FileNotFoundError(f"Missing file: {path}")  
  
    data = pd.read_csv(path, index_col=0)  
    data.index = pd.to_datetime(data.index, errors="coerce")  
    data = data.loc[~data.index.isna()]  
    return data.sort_index()
```

- 下划线开头表示“模块内部使用”；但不是强制隐藏，仍能够被外部调用。
- 对于大型问题，一般会在一个脚本内用多个函数进行功能拆分处理，以保证代码的可读性。

合并上海与深圳市场

```
sh = _read_market(field, "sh", data_dir)
sz = _read_market(field, "sz", data_dir)

data = pd.concat([sh, sz], axis=1)
data = data.sort_index()
data = data.loc[~data.index.duplicated(keep="last")]
data = data.loc[start:end]
```

- `axis=1`: 沿列方向合并两个市场。
- 日期区间使用左右都包含的闭区间。
- 返回结果之前，应保证日期已排序且无重复。

可变数量参数: `*args` 与 `**kwargs`

```
def show_call(*args, **kwargs):  
    print("args:", args)  
    print("kwargs:", kwargs)  
  
show_call("close", "2020-01-01",  
         market="sh", clean=True)  
  
# args: ('close', '2020-01-01')  
# kwargs: {'market': 'sh', 'clean': True}
```

- `*args` 把多余的**位置参数**收集为元组 `tuple`。
- `**kwargs` 把多余的**关键字参数**收集为字典 `dict`。
- `args` 和 `kwargs` 是惯用名; 起作用的是 `*` 与 `**`。

函数还是类?

优先使用函数

- 一次输入对应一次输出;
- 不需要记住之前的状态;
- 操作少且接口简单。

考虑使用类

- 多个操作共享同一配置;
- 需要保存状态;
- “数据 + 行为” 构成对象。

一个简单的数据读取类

```
class MarketDataLoader:
    def __init__(self, data_dir,
                  start_date, end_date):
        self.data_dir = Path(data_dir)
        self.start_date = start_date
        self.end_date = end_date
    def __call__(self, field):
        return data_load(
            field=field,
            start_date=self.start_date,
            end_date=self.end_date,
            data_dir=self.data_dir,
        )
```

对象保存数据目录和固定日期区间；`__call__` 接收要载入的 `field`。

用可调用对象载入不同字段

```
loader = MarketDataLoader(  
    data_dir="dataset",  
    start_date="2020-01-01",  
    end_date="2020-12-31",  
)  
  
close = loader("close")  
open_ = loader("open")  
  
fields = ["close", "open", "high", "low"]  
dataset = {field: loader(field) for field in fields}
```

`loader(field)` 会调用 `loader.__call__(field)`; 每次只需指定字段名。

从脚本到模块和包

- **模块 (module)** : 一个 `.py` 文件。
- **普通包 (package)** : 本课程使用带 `__init__.py` 的目录。
- 模块化的目标是让代码的职责和依赖关系更清楚。

实验项目的目录结构

```
experiment1_template/  
|-- main.py  
|-- self_check.py  
|-- README.md  
`-- data_io/  
    |-- __init__.py  
    `-- io_framework.py
```

- `io_framework.py`: 读取、合并、写出数据。
- `main.py`: 只组织调用, 不重复实现读写逻辑。
- `self_check.py`: 用小型模拟数据检查主要功能。

用 `__init__.py` 定义对外接口

```
from .io_framework import data_load, data_write  
  
__all__ = ["data_load", "data_write"]
```

调用方可以写成：

```
from data_io import data_load, data_write
```

只导出希望使用者直接调用的名称。

主程序入口

```
from data_io import data_load, data_write

if __name__ == "__main__":
    close = data_load(
        "close", "2020-01-01", "2020-12-31"
    )
    print(close.shape)
    data_write(close, "output/close.csv")
```

- 直接运行 `main.py` 时，顺序执行载入、显示和写出。
- 导入 `main.py` 时，跳过该代码块，不自动读写数据。
- `main.py` 只是约定的命名方式，可以按照你的喜好更名，比如叫 `zhangsan.py`。
- 主流程显式展开，方便查看和 debug。（注意：AI 很喜欢用 `main()` 封装，不建

写出数据时的约定

```
def data_write(data, output_file):  
    output_file = Path(output_file)  
    output_file.parent.mkdir(parents=True, exist_ok=True)  
  
    result = data.copy()  
    result.index.name = "date"  
    result.to_csv(output_file, date_format="%Y-%m-%d")
```

- 使用`copy()`命令不会意外修改调用者传入的 DataFrame。
- 输出路径由调用者决定。

函数测试

以 `data_load` 举例

正常输入：两个市场能否正确合并？

边界：开始和结束日期是否被包含？

空结果：区间内没有数据时如何表现？

错误输入：日期反转、文件缺失时是否明确报错？

写出后重新读取：行列与日期是否保持？

实验 1：数据读写与模块化

必做功能

实现 `data_load` 与 `data_write`，将它们组织在 `data_io` 包中，并通过给定的自检程序。

- 读取 `field_sh.csv` 与 `field_sz.csv`;
- 解析、清理、排序日期索引;
- 沿列合并两个市场，并截取闭日期区间;
- 将 `DataFrame` 按指定路径写出为 CSV。

实验报告

提交以下材料:

- `data_io/io_framework.py`
- `data_io/__init__.py`
- `main.py`
- 实验报告, 不超过 2 页。

要求

- 可在项目根目录运行;
- 导入时没有副作用;
- 功能与异常符合约定;
- `self_check.py` 运行通过。

本章小结

- 函数把重复流程封装为稳定接口。
- 先设计输入、输出和错误，再实现函数体。
- 类适合封装需要被记住的配置或状态。
- 模块和包用来划分职责，不是单纯拆分文件。
- 能运行只是起点；还要可读、可导入、可测试。